Dylan Renard

EN.605.617 Introduction to GPU Programming (JHU)

Professor Chance Pascale

February 7th 2026

CUDA Threads and Blocks Assignment

Matrix Addition with Picture Frame Branching (CUDA Threads & Blocks)

Overview

This repo compares **CPU vs GPU** performance for the **same algorithm** (element-wise matrix addition) and studies how CUDA **execution configuration** (blocks / threads-per-block) and **conditional branching** affect runtime.

Two versions of the computation are implemented on both CPU and GPU:

1. **Baseline (minimal branching)**
   (C[i] = A[i] + B[i]) for all elements
2. **Branched ("picture frame" border exclusion)**
   If element is in the **interior region**, compute (A+B); otherwise (border), copy (A).
   This models common image-processing logic and introduces **warp divergence** on the GPU.

Problem Setup

- Matrix size: **2048 × 2048**
- Total elements: **N = 2048 × 2048 = 4,194,304 (2^{22})**
- Fixed border thickness: **32**
- Data type: `float`
- GPU kernels use a **grid-stride loop** so all N elements are processed even when `blocks × threads < N`.

Project Structure (key items)

```
assignment.cu            # CUDA + CPU implementations + timing + guardrails
Makefile                 # Builds assignment.exe
charts.py                # Reads sweep_results.csv and writes figures
charts_out/              # All generated charts and heatmaps (PNG)
sweep_launch_configs.py  # Python script to run many tests of the assignment code.
sweep_out/               # Stores all exploratory run logs from sweep_launch_configs.py
sweep_results.csv        # Output from parameter sweep script (blocks × threads)
```

```
RunLogs/                # timing_results_<blocks>_<threads>.txt from individual runs
README.md               # This file
```

> Note: the repo stores plots in **charts_out/**. The image links below assume that location.

Build

```
make
```

Produces: `assignment.exe`

Run

```
./assignment.exe <num_blocks> <threads_per_block>
```

Required example (spec):

```
./assignment.exe 512 256
```

Output behavior (what prints)

At runtime the program prints: - GPU device name and device limits (max threads/block, max grid X dimension) - Final (possibly clamped) launch configuration: - Blocks - Threads per block - Total threads launched - Timings (nanoseconds): - GPU baseline - GPU branched - CPU baseline - CPU branched - The output filename written (e.g., `timing_results512_256.txt`)

Guardrails (malformed / extreme CLI inputs)

The program defensively handles bad inputs: - Missing args → defaults `(512, 256)` - Non-numeric / zero / negative → clamped to `1` - `threads_per_block` > device max → clamped to `maxThreadsPerBlock` - Extremely large launches (e.g., `100000000 256`) → blocks are clamped using a **conservative upper bound** of `~10×N` total threads to avoid overflow and wasted work

---

Timing Methodology (what the numbers mean)

- CPU timing: `std::chrono::high_resolution_clock` around the full CPU loop
- GPU timing: wall-clock timing around the kernel launch **plus** `cudaDeviceSynchronize()`
  This ensures the measured interval includes **kernel execution** (CUDA launches are otherwise async).

- A warm-up kernel runs once before timing to avoid first-launch CUDA context/JIT overhead.

  Note: `std::chrono` GPU timing includes small host-side overhead. CUDA events would isolate device time more precisely, but this is sufficient for the assignment's goals (configuration comparison + branching impact).

Parameter Sweep

A full grid search was run over 8×8 configurations:

- **Blocks:** `1, 4, 16, 64, 256, 1024, 4096, 16384`
- **Threads per block:** `8, 16, 32, 64, 128, 256, 512, 1024`

Each combination produced timings recorded into `sweep_results.csv`. `charts.py` generates all plots into `charts_out/` excluding the heatmaps which are generated from `sweep_launch_configs.py`.

To reproduce charts:

```
python3 charts.py
```

Results and Analysis (from charts_out charts)

Below are the key takeaways, tied directly to the generated figures.

1) GPU runtime depends strongly on launch configuration

Heatmaps: GPU baseline and GPU branched

- **GPU baseline heatmap:** `charts_out/heat_gpu_baseline.png`
- **GPU branched heatmap:** `charts_out/heat_gpu_branched.png`

These heatmaps show that **GPU runtime drops dramatically** as launch size increases from tiny configurations.

Why: with too few blocks/threads, the GPU is underutilized (many SMs idle). Even though the kernel uses a grid-stride loop (so it still completes), a small number of threads must "walk" the array, reducing parallelism.

As blocks and threads increase: - more warps are resident - occupancy improves - memory latency is hidden better - runtime approaches a **plateau** (diminishing returns once the GPU is saturated)

2) "Good" thread block sizes cluster around warp-friendly ranges

Runtime vs threads (blocks fixed)

- `charts_out/runtime_vs_threads_blocks256.png`

For a fixed block count, increasing threads-per-block typically improves performance from very small values (8–32) up into moderate values (128–256). After that, performance tends to **plateau**.

Interpretation:
- Very small thread blocks do not create enough warps for efficient scheduling. - Moderate thread blocks (often 128–256) usually balance occupancy and scheduling well. - Very large thread blocks (512–1024) can reduce flexibility (e.g., resource limits / register pressure) and don't necessarily speed things up further.

3) Branching slows the GPU, but helps the CPU (expected!)

Heatmaps + runtime plots

- GPU baseline vs branched heatmaps: `heat_gpu_baseline.png`, `heat_gpu_branched.png`
- Runtime vs blocks: `charts_out/runtime_vs_blocks_threads128.png`
- Runtime vs threads: `charts_out/runtime_vs_threads_blocks256.png`

GPU: the branched kernel is generally slower than the baseline across most configurations, with the branching penalty varying by launch configuration and occasionally approaching (or slightly dipping below) parity due to warp alignment, occupancy effects, and measurement noise. **CPU:** branched loop is often faster than CPU baseline because it skips work on the border.

This "opposite" behavior is one of the main lessons:

- On CPU, a branch that skips work can reduce runtime (and branch predictors can help).
- On GPU, branching can cause **warp divergence**: within a warp, some threads take the "interior" path and others take the "border" path, forcing serialization of paths.

Overall takeaway
The CPU benefits from skipping border work, while the GPU pays a divergence penalty because warps execute both branch paths when threads disagree.

Why a thin, fixed border still causes measurable GPU divergence

Although the picture-frame border is fixed at **32 pixels** and represents only a small fraction of the total matrix (~6% of elements), it still produces a consistent

performance penalty on the GPU. This is because warp divergence depends not on how many elements branch overall, but on **how branch conditions align within individual warps**.

Near the boundary between the border and interior regions, warps may contain a mix of threads that take different control-flow paths. When this occurs, the warp must execute both paths serially, masking inactive threads on each path and reducing effective parallelism.

Because the mapping of matrix elements to warps depends on the **block size, thread count, and grid-stride pattern**, the observed branching penalty varies with execution configuration, even though the border thickness is fixed across all experiments.

4) Branching penalty varies with configuration (divergence + occupancy interaction)

Branch penalty plots (ratio branched/baseline)

- `charts_out/gpu_branch_penalty_vs_blocks_threads128.png`
- `charts_out/gpu_branch_penalty_vs_threads_blocks256.png`

The branching penalty is not a single constant value; it changes with launch configuration.

Why it varies: - Divergence occurs primarily near the border region - How border elements map onto warps depends on indexing and scheduling - Occupancy affects how much divergence cost can be hidden

Practical takeaway:
> Even for the same algorithm and data, GPU control-flow cost depends on how the work is scheduled (blocks/threads).

5) Speedup: GPU is consistently faster, but branching reduces speedup

Speedup plots

- `charts_out/speedup_vs_blocks_threads128.png`
- `charts_out/speedup_vs_threads_blocks256.png`

These plots show CPU/GPU speedup for baseline and branched kernels.

Typical pattern: - Speedup rises quickly as GPU configuration improves (more parallelism) - Speedup plateaus once GPU reaches near-saturation - Branched speedup is lower than baseline speedup due to divergence

This demonstrates the assignment's target concept clearly: - **threads/blocks tuning matters** - **branching matters** - GPU remains faster for this data-parallel problem, but the gap shrinks under divergence

6) CPU runtime is mostly constant; fluctuations are measurement noise

CPU heatmaps

- `charts_out/heat_cpu_baseline.png`
- `charts_out/heat_cpu_branched.png`

The CPU algorithm does not use CUDA launch parameters, so it should not systematically depend on blocks/threads.

Small fluctuations appear due to: - OS scheduling / background processes - CPU turbo/frequency scaling - cache state and measurement noise

Overall Takeaway:
> CPU timings vary slightly across runs because we measure wall-clock time on a multitasking OS, not because the CPU algorithm changes with CUDA parameters.

Addressing the example code from last year:

Overall, this submission correctly demonstrates basic CUDA concepts such as device memory allocation, kernel launch syntax, and user-configurable execution parameters. However, it contains a critical benchmarking flaw: the GPU kernel is timed without a synchronization point, so the reported "GPU time" measures only kernel launch overhead rather than actual execution time.

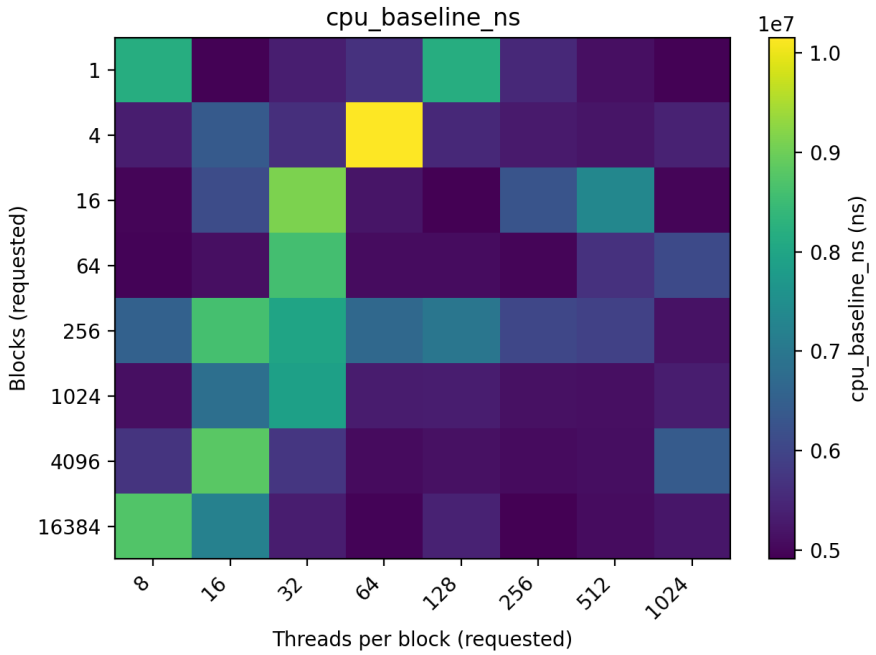
```
int main(int argc,char* argv[]) {

    outputCardInfo();
    // GOOD: Prints GPU/device information, which provides useful context
    // for interpreting performance results.

    int blocks = 3;
    int threads = 64;
    // GOOD: Provides default launch parameters.
    // BAD: These defaults may severely underutilize the GPU for large N.

    if (argc == 2) {
        blocks = atoi(argv[1]);
        printf("Blocks changed to:%i\n", blocks);
        // GOOD: Allows user control over block count.
        // BAD: No validation (negative, zero, or excessively large values).
    }
    else if (argc == 3) {
        blocks = atoi(argv[1]);
        threads = atoi(argv[2]);
        printf("Blocks changed to:%i\n", blocks);
        printf("Threads changed to:%i\n", threads);
```

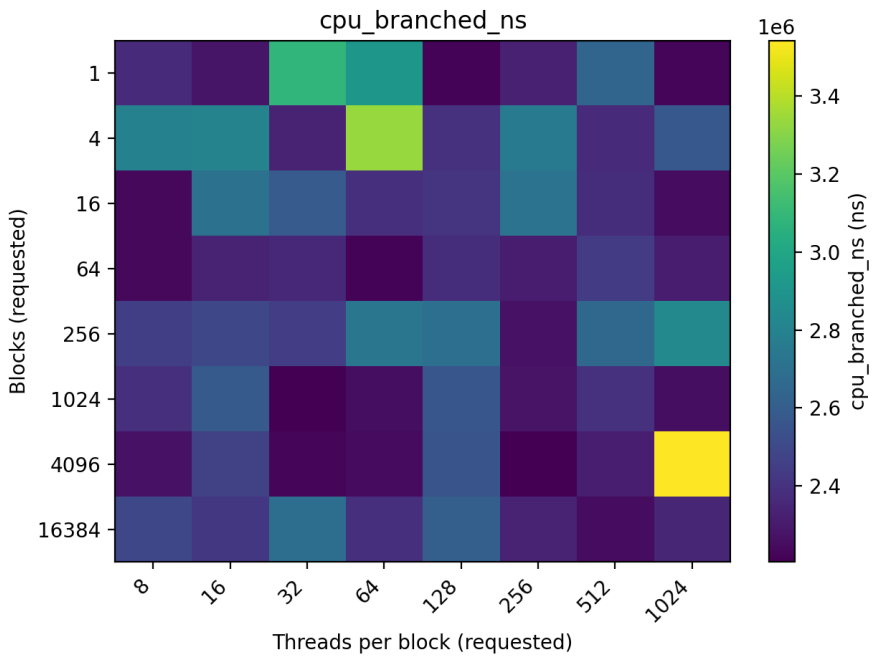
```
    // GOOD: User-configurable execution configuration.
    // BAD: No checks against device limits (max threads per block).
}

int a[N], b[N], c[N];
// GOOD: Simple host-side arrays for demonstration.
// BAD: Large stack allocations can be unsafe for large N.

int *dev_a, *dev_b, *dev_c;
cudaMalloc((void**)&dev_a, N * sizeof(int));
cudaMalloc((void**)&dev_b, N * sizeof(int));
cudaMalloc((void**)&dev_c, N * sizeof(int));
// GOOD: Correct use of CUDA device memory allocation.
// BAD: No CUDA error checking (cudaGetLastError / cudaPeekAtLastError).

// Populate our arrays with numbers.
for (int i = 0; i < N; i++) {
    a[i] = -i;
    b[i] = i*i;
}
// GOOD: Deterministic initialization simplifies debugging and validation.

cudaMemcpy(dev_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(dev_b, b, N * sizeof(int), cudaMemcpyHostToDevice);
// GOOD: Proper host-to-device data transfer.
// BAD: No error checking on cudaMemcpy.

auto start = std::chrono::high_resolution_clock::now();
add<<<blocks,threads>>>(dev_a, dev_b, dev_c);
auto stop = std::chrono::high_resolution_clock::now();
// BAD (CRITICAL): Kernel launches are asynchronous.
// Timing stops before the kernel actually finishes executing.
// This measures launch overhead, not computation time.

cudaMemcpy(c, dev_c, N*sizeof(int), cudaMemcpyDeviceToHost);
// BAD (TIMING ISSUE): This call synchronizes the GPU,
// but its cost is NOT included in the GPU timing above.

cudaFree(dev_a);
cudaFree(dev_b);
cudaFree(dev_c);
// GOOD: Proper cleanup of device memory.

auto startHost = std::chrono::high_resolution_clock::now();
addHost(a, b, c);
auto stopHost = std::chrono::high_resolution_clock::now();
```

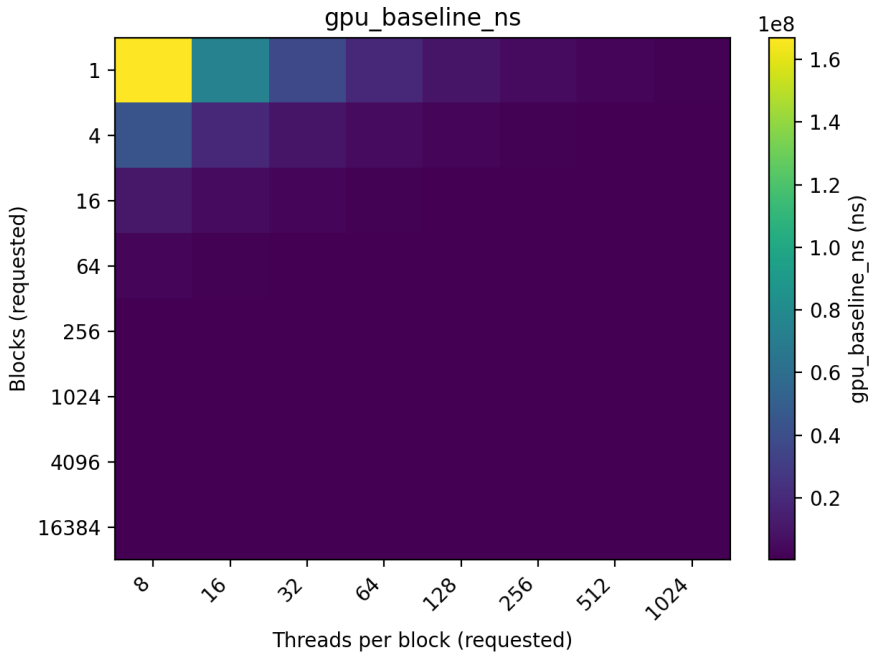
```
    // GOOD: CPU timing correctly captures full execution time.

    std::cout << "\n Time elapsed GPU = "
              << std::chrono::duration_cast<std::chrono::nanoseconds>(stop - start).count()
              << " ns\n";
    std::cout << " Time elapsed Host = "
              << std::chrono::duration_cast<std::chrono::nanoseconds>(stopHost - startHost).
              << " ns\n";
    // BAD: GPU and CPU timings are not comparable due to incorrect GPU timing.

    return 0;
}
```

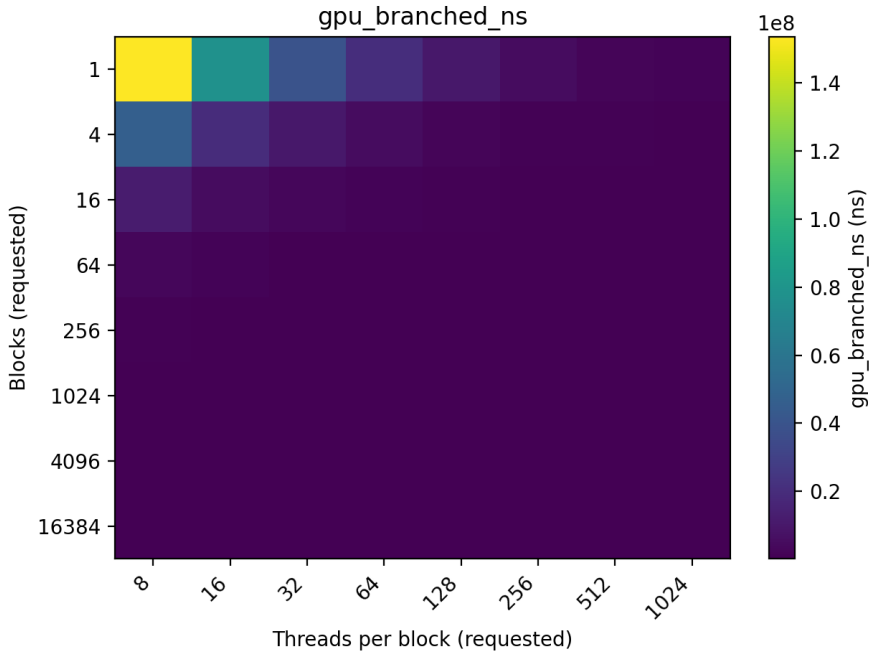
Figures (embedded)

Heatmaps

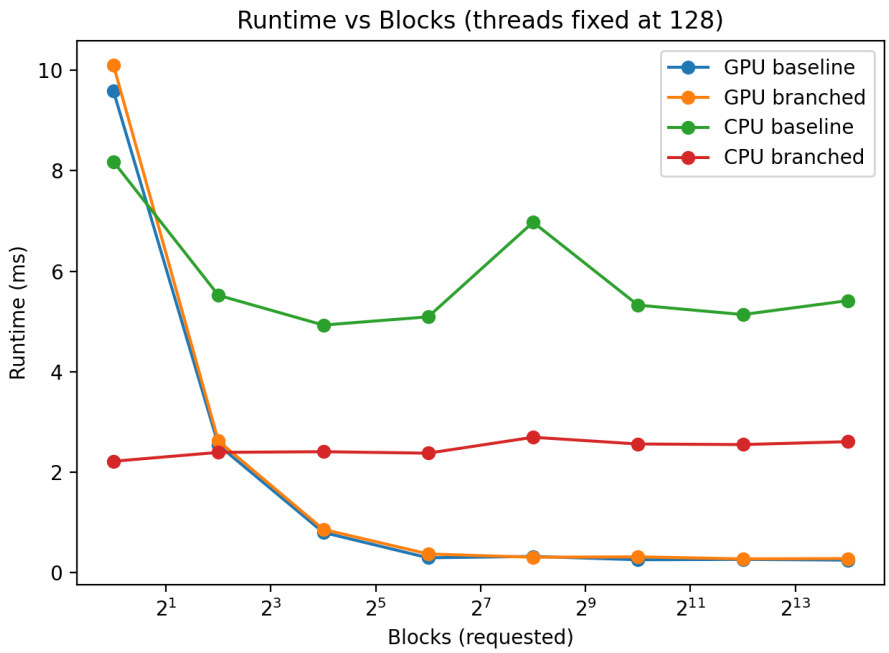


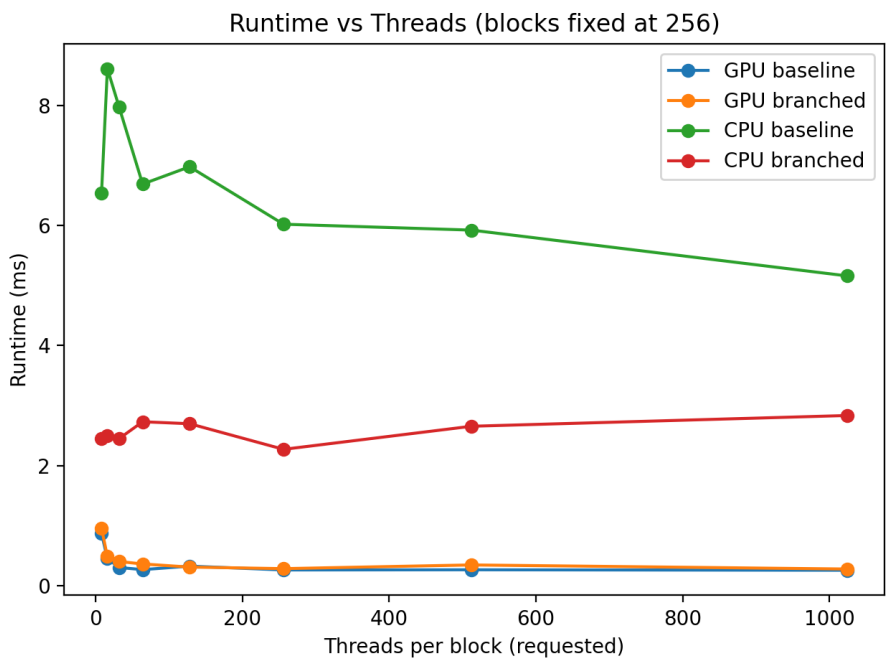



gpu_baseline_ns
1e8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
gpu_baseline_ns (ns)
Blocks (requested)
1
4
16
64
256
1024
4096
16384
8
16
32
64
128
256
512
1024
Threads per block (requested)

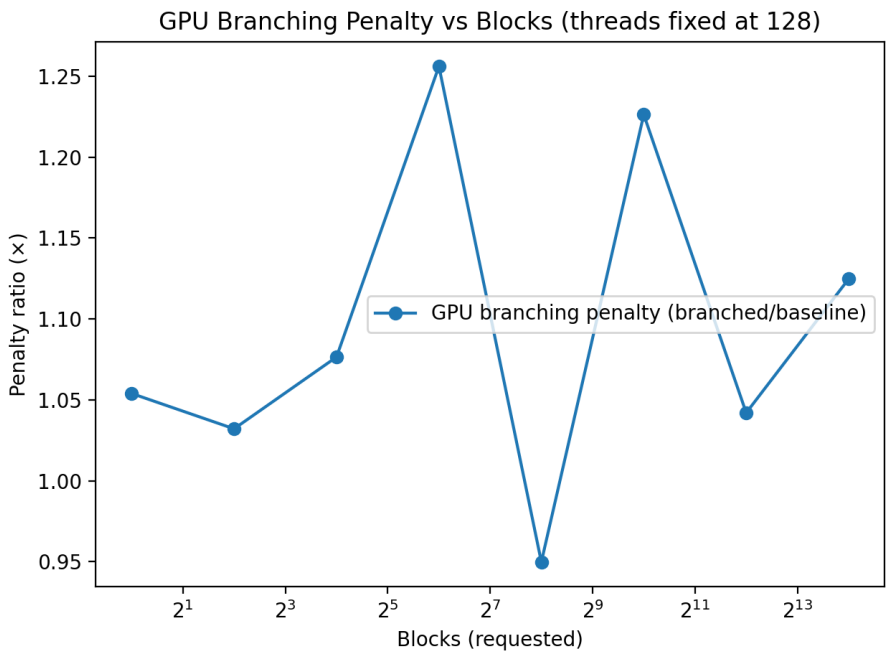

gpu_branched_ns
1e8
1.4
1.2
1.0
0.8
0.6
0.4
0.2
gpu_branched_ns (ns)
Blocks (requested)
1
4
16
64
256
1024
4096
16384
8
16
32
64
128
256
512
1024
Threads per block (requested)

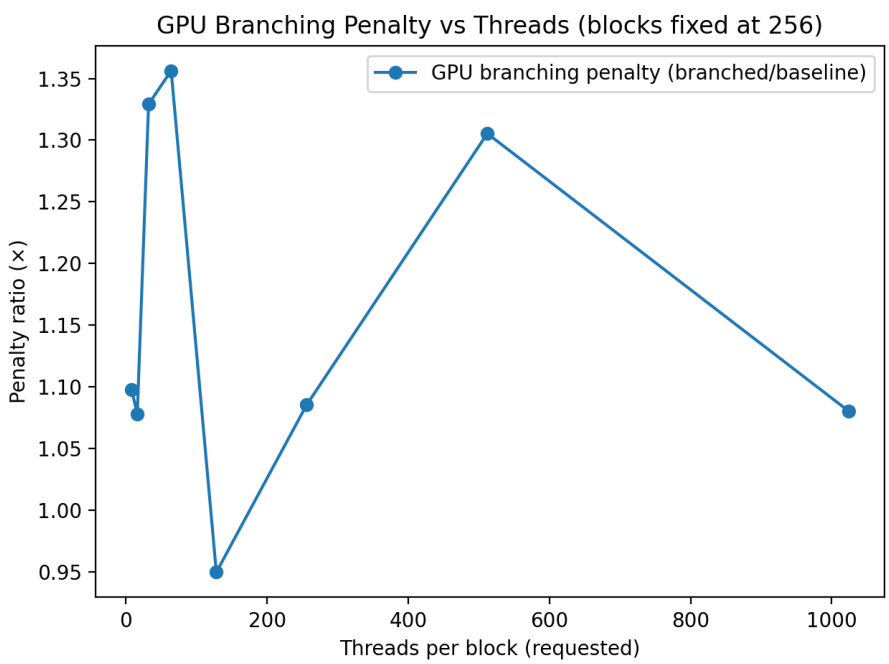
Runtime trends



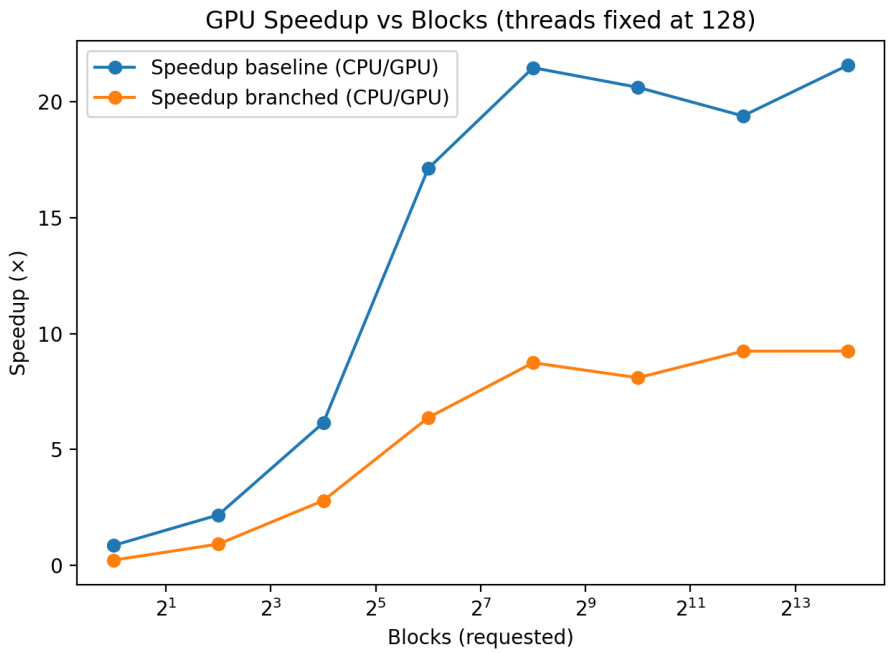


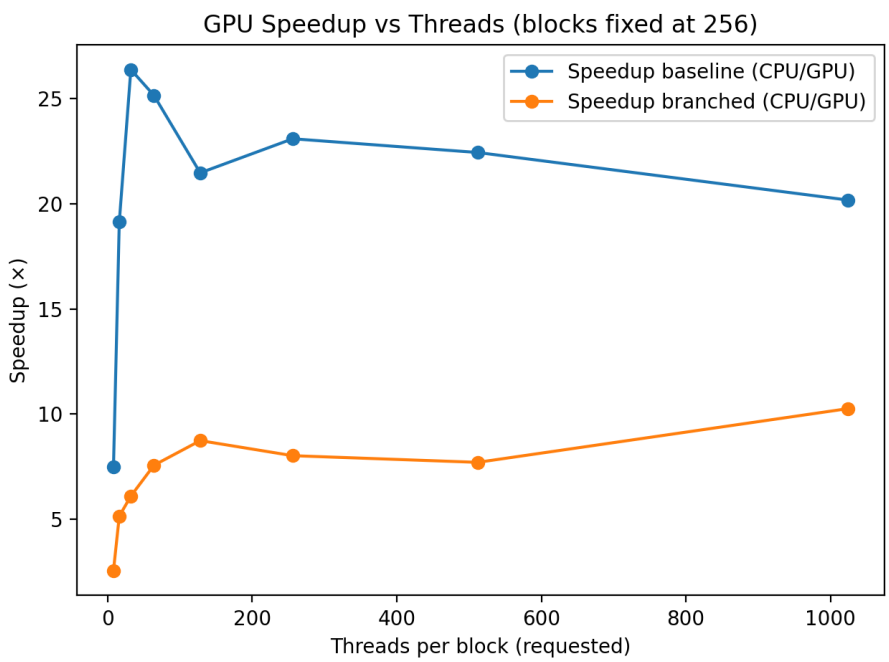
Divergence / branching penalty

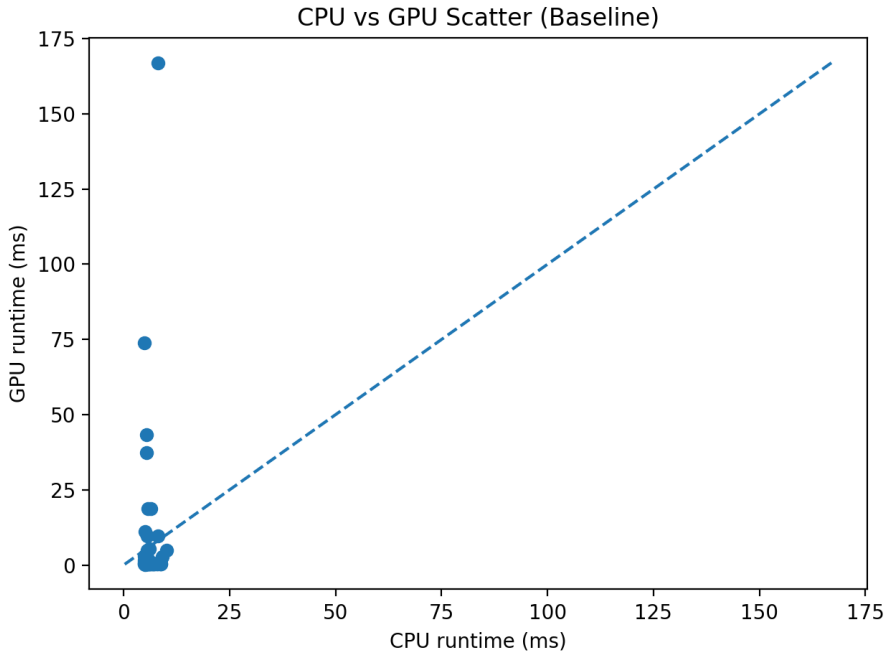


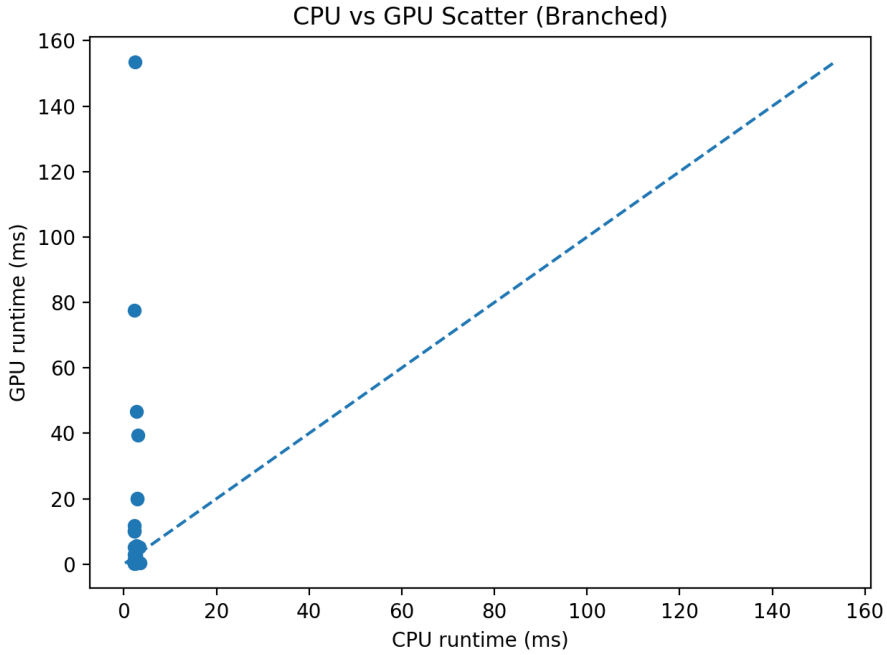


Speedup and CPU vs GPU scatter






CPU vs GPU Scatter (Baseline)
GPU runtime (ms)
CPU runtime (ms)
0
25
50
75
100
125
150
175


CPU vs GPU Scatter (Branched)
GPU runtime (ms)
CPU runtime (ms)
0
20
40
60
80
100
120
140
160